

Algorithmes et Pensée Computationnelle

Fonctions, mémoire et exceptions

Le but de cette séance est d'approfondir vos connaissances en programmation. Au terme de cette séance, l'étudiant sera capable de :

- définir une fonction et l'utiliser dans un programme,
- utiliser des bibliothèques contenant des fonctions prédéfinies,
- connaître quelle est la portée d'une variable,
- comprendre comment fonctionne une pile d'exécution (call stack),
- gérer des exceptions.

1 Opérations arithmétiques

Question 1: (🕒 3 minutes) Calculs (multiplication) (Java ou Python)

Créez deux variables `facteur_1` (= 11) et `facteur_2` (= 3). Multipliez la première variable par la deuxième et stockez le résultat dans une nouvelle variable `produit`. Vous pouvez afficher les différentes variables pour voir leurs valeurs. Vous pouvez répéter l'exercice avec l'addition et la soustraction.

💡 Conseil

L'opérateur de multiplication est le `*`, celui d'addition est le `+` et celui de soustraction est le `-`.

Question 2: (🕒 10 minutes) Calculs (division) (Java ou Python)

Créez deux variables `nb_bonbons` avec pour valeur 11 et `nb_personnes` avec pour valeur 3. Divisez la première variable par la deuxième et stockez le résultat dans une nouvelle variable `bonbons_personnes`. Pour finir, calculez le nombre de bonbons restants via l'opérateur `%` (modulo) et stockez le résultat dans une nouvelle variable `reste`. Vous pouvez afficher les différentes variables pour voir leurs valeurs.

💡 Conseil

- Attention, en Python il existe 2 opérateurs de division, `/` effectue une division classique, tandis que `//` effectue une division entière.
- En Java, si vous travaillez uniquement avec des `int`, `/` effectuera une division entière tandis que si vous travaillez avec au moins un `float`, `/` effectuera une division classique.
- L'opérateur `%` (modulo) permet de calculer le reste d'une division. Par exemple, $10\%2=0$.

Question 3: (🕒 5 minutes) Calculs (incréméntation / décrémentation) (Java ou Python)

Gardez vos variables de l'exercice précédent, augmentez la valeur de `nb_bonbons` de 1, et diminuez celle de `nb_personnes` de 1.

💡 Conseil

Vous pouvez utiliser les opérateurs `+=` et `-=` en Python, et les opérateurs `++` (incréméntation) et `--` (décrémentation) en Java.

2 Variables et Fonctions

Question 4: (🕒 5 minutes) Les fonctions (fonctions basiques) (Java et Python)

Définissez une fonction nommée `ping()` qui, lorsqu'elle est appelée, affiche "pong".

💡 Conseil

Référez vous aux diapositives du cours pour la création et l'appel des fonctions.

Question 5: (🕒 5 minutes) Les Fonctions (Fonction multiplication) (Java et Python)

Définissez une fonction nommée `multiplicateur()` qui prend deux arguments *multiple1* et *multiple2*, les multiplie et retourne le résultat. Stockez le résultat de `multiplicateur(2, 3)` dans une variable *resultat* et affichez la.

💡 Conseil

- Référez vous aux diapositives du cours pour la création et l'appel des fonctions.
- Pour retourner une valeur au lieu de l'afficher, utilisez le mot-clé `return` (pour Python et Java).

Question 6: (🕒 5 minutes) Les Fonctions (Fonctions Aire et Périmètre) (Java et Python)

Définissez deux fonctions nommées `aire()` et `perimetre()` qui prennent un argument (*rayon*) et renvoient respectivement l'aire et le périmètre d'un cercle. Stockez les résultats dans des variables *aire* et *perimetre* et affichez le contenu de ces variables.

💡 Conseil

- Référez vous aux diapositives du cours pour la création et l'appel des fonctions.
- Pour retourner une valeur au lieu de l'imprimer, utilisez le mot clé `return` (pour Python et Java).
- Pour rappel, le périmètre d'un cercle s'obtient en utilisant la formule $P = 2 * \pi * r$ et l'aire s'obtient en utilisant la formule $A = r^2 * \pi$.

Question 7: (🕒 5 minutes) Portée des variables (Python)

Qu'affiche le programme suivant ?

```
1 x = 2
2
3 def fonction():
4     x = 3
5
6     def fonction2():
7         global x
8         print("x=" + str(x))
9
10    fonction2()
11    print("x=" + str(x))
12
13 print(fonction())
```

Conseil

- Le mot-clé `global` permet d'accéder aux variables globales (définies à l'extérieur de la fonction).
- Soyez attentif au type des éléments retournés par chacune des fonctions.

Question 8: (🕒 10 minutes) Portée des variables (Java)

Qu'affiche le programme suivant ?

```
1 public class Main {
2     static int a = 0;
3     public static void f() {
4         a += 2;
5         System.out.println("a = " + a);
6     }
7
8     public static void g() {
9         int b = 3;
10        System.out.println("a = " + a);
11        System.out.println("b = " + b);
12    }
13
14    public static void main(String[] args) {
15        f();
16        g();
17        //System.out.println("b = " + b);
18    }
19 }
```

Que se passerait-il si on décommente la ligne 17 (enlever les //) ?

Conseil

Tenir compte de l'endroit où chaque variable est définie et quelles opérations sont réalisées sur celle-ci.

3 Gestion des exceptions

Question 9: (🕒 5 minutes) Types d'erreurs (Python) Qu'affiche le programme suivant ?

```
1 value1 = "Algorithms"
2 value2 = 4
3
4 try:
5     size = len(value1)
6     result = size/value2
7     print(f"Le résultat de la division est: {result}")
8
9 except Exception as error:
10    print("On ne peut pas effectuer l'opération")
11
12 try:
13     result = value1/2
14     print(f"Le résultat de la division est: {result}")
15
```

```

16 except TypeError as error:
17     print("On ne peut pas diviser une chaîne de caractères")

```

Conseil

Utilisée sur une chaîne de caractères, la fonction `len()` renvoie le nombre de caractères de la chaîne.

Question 10: (🕒 5 minutes) **Types d'erreurs (Python)** Qu'affiche le programme suivant ?

```

1 value1 = 4
2 value2 = ""
3
4 try:
5     count = len(value2)
6     result= value1/count
7 except ZeroDivisionError as error:
8     print("Nous ne pouvons pas diviser un nombre par 0")

```

Conseil

La chaîne de caractères vide est représentée par `""`

Question 11: (🕒 5 minutes) **Types d'erreurs (Python)** Qu'affiche le programme suivant ?

```

1 value1 = "Algorithms"
2 value2 = 4
3
4
5 try:
6     decimal = float(value2)
7 except ValueError as error:
8     print("Nous ne pouvons pas convertir un entier en décimal")
9
10 finally:
11     try:
12         value2 = int(value1)
13     except ValueError as error:
14         print("Nous ne pouvons pas convertir une chaîne de caractères
    en nombre")

```

Conseil

- La fonction `float()` permet de convertir une variable en nombre décimal.
- La fonction `int()` permet de convertir une variable en nombre entier.

Question 12: (🕒 5 minutes) **Gestion d'erreurs (Java)** Le programme suivant est incorrect. Que devez-vous modifier pour qu'il fonctionne correctement ?

Conseil

Référez-vous à la diapositive 24 du cours de cette semaine.

```

1 public class Question9 {
2
3     public static void division(int a, int b) throws
        ArithmeticException{
4         if(b==0){
5             throw new ArithmeticException();
6         }else{
7             float result = a/b;
8             System.out.println("Le résultat de la division de " + a +
                "/" + b + " = " + result);
9         }
10    }
11    public static void main(String args[]){
12        int value1 = 2;
13        int value2 = 4;
14        try {
15            division(value1,value2);
16            value2 = 0;
17            division(value1,value2);
18        }catch(IndexOutOfBoundsException err){
19            System.out.println("Nous ne pouvons pas effectuer une
                division par 0");
20        }
21    }
22 }

```

Question 13: (🕒 5 minutes) Gestion d'erreurs (Java)

1. Qu'affiche le programme suivant supposant que l'utilisateur saisit l'entrée suivante ?

- Entrez le premier entier : 12
- Entrez le deuxième entier : 2
- Entrez l'opération (+, -, *, /): *

💡 Conseil

Consultez [ce document](#) sur le switch en Java. Référez-vous à la diapositive 23 du cours.

2. Que devez-vous modifier pour qu'il fonctionne ?

```

1 import java.util.Scanner;
2
3 public class Calculator {
4     public static void main(String[] args) {
5         Scanner sc = new Scanner(System.in);
6
7         System.out.print("Entrez le premier entier: ");
8         double num1 = sc.nextInt();
9
10        System.out.print("Entrez le deuxième entier: ");
11        double num2 = sc.nextInt();
12
13        System.out.print("Entrez l'opération (+, -, *, /): ");
14        char op = sc.next().charAt(0);
15
16        sc.close();
17
18        switch (op) {

```

```

19         case '+':
20             System.out.println(num1 + num2);
21         case '-':
22             System.out.println(num1 - num2);
23         case '*':
24             System.out.println(num1 * num2);
25         case '/':
26             System.out.println(num1 / num2);
27         default:
28             throw new IllegalArgumentException("L'opération n'est
pas valide");
29     }
30 }
31 }

```

Question 14: (🕒 5 minutes) Qu'affiche le programme (question11.py) suivant supposant que l'utilisateur exécute le programme de la manière suivante dans le terminal ?

```
python3 question11.py 9
```

💡 Conseil

Référez-vous à la diapositive 10 du cours.

```

1 import sys
2 if __name__ == '__main__':
3     balance = 20
4     try:
5         balance = balance - sys.argv[1]
6         solde_post = balance > 0
7         match (solde_post):
8             case True:
9                 print('Le solde de votre compte est positif.')
10            case False:
11                print('Le solde de votre compte est négatif ou zéro.')
12                raise ValueError('Le compte ne peut pas être débité')
13 except Exception as e:
14     print("Une erreur est survenue")
15 finally:
16     print(f"Valeur finale de balance: {balance}")

```

Question 15: (🕒 5 minutes) Laquelle des affirmations suivantes est **fausse** concernant la différence entre une Error (erreur) et une Exception (exception) en Java ?

💡 Conseil

Référez-vous à la diapositive 34 du cours.

- (A) Les erreurs sont généralement causées par l'environnement dans lequel l'application s'exécute, tandis que les exceptions sont généralement causées par l'application elle-même.
- (B) Les erreurs sont non vérifiées (*unchecked*) et n'ont pas besoin d'être déclarées dans la clause `throws` d'une méthode, tandis que les exceptions peuvent être soit vérifiées (*checked*), soit non vérifiées (*unchecked*).
- (C) Les erreurs devraient généralement être interceptées et gérées par l'application afin d'assurer une récupération fluide, tandis que les exceptions peuvent être récupérables ou non.

- (D) Les erreurs indiquent généralement des problèmes graves qu'une application raisonnable ne devrait pas essayer d'intercepter.
- (E) `RuntimeException` correspond aux exceptions non vérifiées (*unchecked*).